

SRS: Student–StudentProfile One-to-One Mapping Spring Boot Project

1. Project Title

Student Profile Management System using One-to-One Mapping in Spring Boot

2. Project Objective

Build a Spring Boot REST API project where:

```
One Student has one StudentProfile
One StudentProfile belongs to one Student
```

The project must include:

```
Entity layer
Repository layer
Service layer
Controller layer
DTO layer
Exception handling
Role-based basic security without JWT
Swagger/OpenAPI documentation
Pagination
Validation
ModelMapper
Lombok
MySQL database
```

3. Relationship Requirement

The project must implement a **bidirectional one-to-one mapping** between:

```
Student
StudentProfile
```

Relationship rule:

```
Student 1 ---- 1 StudentProfile
```

Database relationship:

```
student_profiles.student_id → students.id
```

The foreign key must be stored in the `student_profiles` table.

4. Required Technology Stack

Use the following:

```
Java 17 or above
Spring Boot 3.x
Spring Web
Spring Data JPA
Spring Security
Spring Validation
MySQL
Lombok
ModelMapper
Springdoc OpenAPI / Swagger
Maven
Postman
```

5. Required Maven Dependencies

The project must include dependencies for:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-validation
spring-boot-starter-security
mysql-connector-j
modelmapper
springdoc-openapi-starter-webmvc-ui
lombok
```

6. Database Requirement

Create only the database manually:

```
CREATE DATABASE one_to_one_demo;
```

Do not manually create tables.

Hibernate must create/update tables automatically using:

```
spring.jpa.hibernate.ddl-auto=update
```

7. Application Configuration Requirement

File:

```
src/main/resources/application.properties
```

Must contain configuration keys for:

```
spring.datasource.url
spring.datasource.username
spring.datasource.password
spring.jpa.hibernate.ddl-auto
spring.jpa.show-sql
spring.jpa.properties.hibernate.format_sql
server.port
springdoc.swagger-ui.path
```

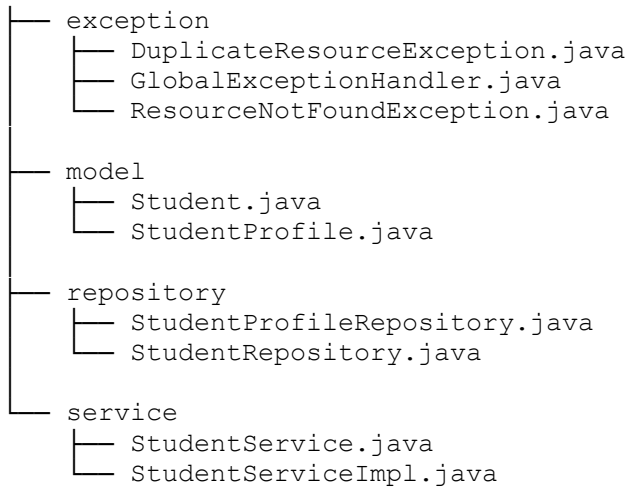
Use port:

```
8080
```

8. Final Project Structure

Create the project structure exactly like this:

```
src/main/java/com/swabhav/demo
├── DemoApplication.java
├── config
│   ├── AppConfig.java
│   ├── SecurityConfig.java
│   └── OpenApiConfig.java
├── controller
│   └── StudentController.java
├── dto
│   ├── PageResponseDto.java
│   ├── StudentProfileRequestDto.java
│   ├── StudentProfileResponseDto.java
│   ├── StudentRequestDto.java
│   └── StudentResponseDto.java
```



9. Package-Wise Instructions

9.1 `model` Package

This package must contain entity classes only.

9.2 `dto` Package

This package must contain request and response DTOs only.

9.3 `repository` Package

This package must contain Spring Data JPA repository interfaces only.

9.4 `service` Package

This package must contain:

Service interface
Service implementation class

Business logic must be written only in the service implementation class.

9.5 `controller` Package

This package must contain REST controller classes only.

9.6 `exception` Package

This package must contain:

Custom exception classes
Global exception handler

9.7 `config` Package

This package must contain:

ModelMapper bean configuration
Spring Security configuration
Swagger/OpenAPI configuration

10. Entity Class Requirements

10.1 `Student.java`

Package:

`com.swabhav.demo.model`

Class type:

Entity class

Table name:

`students`

Required annotations:

```
@Entity
@Table
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

Fields required:

Field Name	Data Type	Column Requirement
id	Long	Primary key, auto-generated
fullName	String	Mapped to full_name, not null
age	Integer	Not null
profile	StudentProfile	One-to-one mapped field

Relationship requirement:

Student must have one StudentProfile.
Student must be the inverse side of the relationship.
StudentProfile must own the foreign key.

Mapping requirement for profile field:

```
@OneToOne
mappedBy = "student"
cascade = CascadeType.ALL
orphanRemoval = true
fetch = FetchType.LAZY
```

No business logic should be added inside this class.

10.2 StudentProfile.java

Package:

```
com.swabhav.demo.model
```

Class type:

```
Entity class
```

Table name:

```
student_profiles
```

Required annotations:

```
@Entity
@Table
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

Fields required:

Field Name	Data Type	Column Requirement
id	Long	Primary key, auto-generated
email	String	Not null, unique
phone	String	Not null
city	String	Not null
student	Student	Owning side of one-to-one mapping

Relationship requirement:

```
StudentProfile must own the foreign key.
Foreign key column must be student_id.
student_id must be unique.
student_id must be not null.
```

Mapping requirement for student field:

```
@OneToOne
@JoinColumn(name = "student_id", nullable = false, unique = true)
fetch = FetchType.LAZY
```

No business logic should be added inside this class.

11. DTO Class Requirements

11.1 StudentProfileRequestDto.java

Package:

```
com.swabhav.demo.dto
```

Purpose:

Used while creating or updating profile data.

Required annotations:

@Data

Fields required:

Field Name	Data Type	Validation
email	String	@NotBlank, @Email
phone	String	@NotBlank
city	String	@NotBlank

No id field should be present in request DTO.

11.2 StudentRequestDto.java

Package:

com.swabhav.demo.dto

Purpose:

Used while creating or updating student with profile.

Required annotations:

@Data

Fields required:

Field Name	Data Type	Validation
fullName	String	@NotBlank
age	Integer	@NotNull, @Min(1)
profile	StudentProfileRequestDto	@Valid, @NotNull

JSON field requirement:

fullName must appear as full_name in JSON.

Use:

@JsonProperty("full_name")

11.3 StudentProfileResponseDto.java

Package:

com.swabhav.demo.dto

Purpose:

Used to send profile details in API response.

Required annotations:

@Data

Fields required:

Field Name	Data Type
id	Long
email	String
phone	String
city	String

11.4 StudentResponseDto.java

Package:

com.swabhav.demo.dto

Purpose:

Used to send student details with profile in API response.

Required annotations:

@Data

Fields required:

Field Name	Data Type
id	Long
fullName	String
age	Integer
profile	StudentProfileResponseDto

JSON field requirement:

fullName must appear as full_name in JSON.

Use:

```
@JsonProperty("full_name")
```

11.5 PageResponseDto.java

Package:

```
com.swabhav.demo.dto
```

Purpose:

Used to return paginated API response.

Required annotations:

```
@Data  
@NoArgsConstructor  
@AllArgsConstructor
```

Generic type:

```
PageResponseDto<T>
```

Fields required:

Field Name	Data Type
content	List<T>
pageNumber	int
pageSize	int
totalElements	long
totalPages	int
lastPage	boolean

12. Repository Interface Requirements

12.1 StudentRepository.java

Package:

`com.swabhav.demo.repository`

Interface type:

Spring Data JPA repository

Must extend:

`JpaRepository<Student, Long>`

Required methods:

No custom method required initially.

12.2 `StudentProfileRepository.java`

Package:

`com.swabhav.demo.repository`

Interface type:

Spring Data JPA repository

Must extend:

`JpaRepository<StudentProfile, Long>`

Required custom methods:

`boolean existsByEmail(String email)`

`boolean existsByEmailAndIdNot(String email, Long id)`

Purpose of methods:

Method	Purpose
<code>existsByEmail</code>	Check duplicate email during create
<code>existsByEmailAndIdNot</code>	Check duplicate email during update, excluding current profile

13. Service Layer Requirements

13.1 `StudentService.java`

Package:

`com.swabhav.demo.service`

Type:

Interface

Purpose:

Define all student-related operations.

Required methods:

`StudentResponseDto createStudent(StudentRequestDto requestDto)`

`List<StudentResponseDto> getAllStudents()`

`PageResponseDto<StudentResponseDto> getAllStudentsWithPagination(int pageNumber, int pageSize)`

`StudentResponseDto getStudentById(Long id)`

`StudentResponseDto updateStudent(Long id, StudentRequestDto requestDto)`

`void deleteStudent(Long id)`

No implementation should be written in this interface.

13.2 `StudentServiceImpl.java`

Package:

`com.swabhav.demo.service`

Type:

Service implementation class

Required annotations:

`@Service`

`@RequiredArgsConstructor`

Must implement:

StudentService

Required dependencies:

Field Name	Type
studentRepository	StudentRepository
studentProfileRepository	StudentProfileRepository
modelMapper	ModelMapper

Fields must be:

private final

Required public methods:

```
StudentResponseDto createStudent(StudentRequestDto requestDto)

List<StudentResponseDto> getAllStudents()

PageResponseDto<StudentResponseDto> getAllStudentsWithPagination(int
pageNumber, int pageSize)

StudentResponseDto getStudentById(Long id)

StudentResponseDto updateStudent(Long id, StudentRequestDto requestDto)

void deleteStudent(Long id)
```

Required private helper methods:

```
Student findStudentById(Long id)

void updateProfile(StudentProfile profile, StudentProfileRequestDto
profileRequestDto)

void validatePagination(int pageNumber, int pageSize)
```

Business rules to implement:

Create Student

Must:

```
Check if profile email already exists.
Map StudentRequestDto to Student.
Map StudentProfileRequestDto to StudentProfile.
Set profile inside student.
Set student inside profile.
```

Save student.
Return StudentResponseDto.

Get All Students

Must:

Fetch all students.
Map each Student to StudentResponseDto.
Return list.

Get Students With Pagination

Must:

Validate pageNumber and pageSize.
Create Pageable object.
Fetch Page<Student>.
Map content to List<StudentResponseDto>.
Return PageResponseDto<StudentResponseDto>.

Get Student By ID

Must:

Find student by id.
If not found, throw ResourceNotFoundException.
Map Student to StudentResponseDto.
Return response.

Update Student

Must:

Find existing student.
Check if new email already exists for another profile.
Update student fullName.
Update student age.
Update existing profile email.
Update existing profile phone.
Update existing profile city.
Maintain both sides of one-to-one relationship.
Save updated student.
Return StudentResponseDto.

Delete Student

Must:

Find student by id.
Delete student.
Profile must also be deleted because of cascade and orphanRemoval.

Pagination Validation

Must validate:

pageNumber must not be negative.
pageSize must be greater than 0.
pageSize must not be greater than 100.

14. Controller Layer Requirements

14.1 `StudentController.java`

Package:

`com.swabhav.demo.controller`

Type:

REST Controller

Required annotations:

```
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
@SecurityRequirement(name = "basicAuth")
```

Required dependency:

Field Name	Type
studentService	StudentService

Field must be:

`private final`

Required APIs:

API 1: Create Student With Profile

Method name:

`createStudent`

HTTP method:

`POST`

URL:

`/api/students`

Request body:

`StudentRequestDto`

Validation:

`@Valid`

Response:

`StudentResponseDto`

Status:

`201 CREATED`

Access:

`ADMIN only`

API 2: Get All Students

Method name:

`getAllStudents`

HTTP method:

`GET`

URL:

`/api/students`

Response:

`List<StudentResponseDto>`

Access:

USER or ADMIN

API 3: Get Students With Pagination

Method name:

`getAllStudentsWithPagination`

HTTP method:

GET

URL:

`/api/students/page`

Request parameters:

Parameter	Type	Default Value
pageNumber	int	0
pageSize	int	5

Response:

`PageResponseDto<StudentResponseDto>`

Access:

USER or ADMIN

API 4: Get Student By ID

Method name:

`getStudentById`

HTTP method:

GET

URL:

/api/students/{id}

Path variable:

Long id

Response:

StudentResponseDto

Access:

USER or ADMIN

API 5: Update Student With Profile

Method name:

updateStudent

HTTP method:

PUT

URL:

/api/students/{id}

Path variable:

Long id

Request body:

StudentRequestDto

Validation:

@Valid

Response:

StudentResponseDto

Access:

ADMIN only

API 6: Delete Student

Method name:

deleteStudent

HTTP method:

DELETE

URL:

/api/students/{id}

Path variable:

Long id

Response:

No body

Status:

204 NO CONTENT

Access:

ADMIN only

15. Exception Handling Requirements

15.1 ResourceNotFoundException.java

Package:

`com.swabhav.demo.exception`

Type:

Custom runtime exception

Must extend:

`RuntimeException`

Required constructor:

`ResourceNotFoundException(String message)`

Purpose:

Throw when student does not exist.

15.2 DuplicateResourceException.java

Package:

`com.swabhav.demo.exception`

Type:

Custom runtime exception

Must extend:

`RuntimeException`

Required constructor:

`DuplicateResourceException(String message)`

Purpose:

Throw when duplicate email exists.

15.3 GlobalExceptionHandler.java

Package:

`com.swabhav.demo.exception`

Type:

Global exception handler

Required annotation:

@RestControllerAdvice

Required exception handler methods:

```
handleResourceNotFound(ResourceNotFoundException ex)
handleDuplicateResource(DuplicateResourceException ex)
handleValidation(MethodArgumentNotValidException ex)
handleTypeMismatch(MethodArgumentTypeMismatchException ex)
handleInvalidJson(HttpMessageNotReadableException ex)
handleDataIntegrity(DataIntegrityViolationException ex)
handleAccessDenied(AccessDeniedException ex)
handleGeneric(Exception ex)
```

Required private helper method:

```
buildResponse(HttpStatus status, String message)
```

Standard error response fields:

```
timestamp
status
error
message
```

Validation error response fields:

```
timestamp
status
error
messages
```

Required status codes:

Exception	Status
ResourceNotFoundException	404 NOT FOUND
DuplicateResourceException	409 CONFLICT
MethodArgumentNotValidException	400 BAD REQUEST
MethodArgumentTypeMismatchException	400 BAD REQUEST

Exception	Status
HttpMessageNotReadableException	400 BAD REQUEST
DataIntegrityViolationException	409 CONFLICT
AccessDeniedException	403 FORBIDDEN
Generic Exception	500 INTERNAL SERVER ERROR

16. Configuration Class Requirements

16.1 AppConfig.java

Package:

`com.swabhav.demo.config`

Purpose:

Create `ModelMapper` bean.

Required annotation:

`@Configuration`

Required method:

`ModelMapper modelMapper()`

Return type:

`ModelMapper`

Annotation on method:

`@Bean`

16.2 SecurityConfig.java

Package:

`com.swabhav.demo.config`

Purpose:

Configure role-based Basic Authentication without JWT.

Required annotation:

```
@Configuration
```

Required methods:

```
SecurityFilterChain securityFilterChain(HttpSecurity http)
```

```
UserDetailsService userDetailsService(PasswordEncoder passwordEncoder)
```

```
PasswordEncoder passwordEncoder()
```

Required users:

Username	Password	Role
----------	----------	------

admin	admin123	ADMIN
-------	----------	-------

user	user123	USER
------	---------	------

Security rules:

HTTP Method	URL Pattern	Access
GET	/api/students/**	USER or ADMIN
POST	/api/students/**	ADMIN only
PUT	/api/students/**	ADMIN only
DELETE	/api/students/**	ADMIN only

Swagger endpoints must be public:

```
/swagger-ui.html  
/swagger-ui/**  
/v3/api-docs/**
```

Other security requirements:

CSRF must be disabled.

HTTP Basic authentication must be enabled.

Session policy must be stateless.

Password must be encoded using BCryptPasswordEncoder.

16.3 OpenApiConfig.java

Package:

```
com.swabhav.demo.config
```

Purpose:

Configure Swagger/OpenAPI documentation.

Required annotation:

@Configuration
@OpenAPIDefinition
@SecurityScheme

OpenAPI info must include:

Property	Value
Title	Student One-to-One Mapping API
Version	1.0
Description	Spring Boot REST API with One-to-One JPA mapping, DTOs, Security, and Swagger

Security scheme:

Name: basicAuth
Type: HTTP
Scheme: basic

17. Main Application Class Requirement

DemoApplication.java

Package:

com.swabhav.demo

Required annotation:

@SpringBootApplication

Required method:

main(String[] args)

Purpose:

Start the Spring Boot application.

18. API Request and Response Requirements

18.1 Create Student Request

Endpoint:

POST /api/students

Request body:

```
{
  "full_name": "Rahul Sharma",
  "age": 21,
  "profile": {
    "email": "rahul@gmail.com",
    "phone": "9876543210",
    "city": "Mumbai"
  }
}
```

Expected response fields:

```
id
full_name
age
profile.id
profile.email
profile.phone
profile.city
```

18.2 Update Student Request

Endpoint:

PUT /api/students/{id}

Request body:

```
{
  "full_name": "Rahul Kumar Sharma",
  "age": 22,
  "profile": {
    "email": "rahul.updated@gmail.com",
    "phone": "9999999999",
    "city": "Pune"
  }
}
```

```
}  
}
```

18.3 Pagination Response

Endpoint:

GET /api/students/page?pageNumber=0&pageSize=5

Expected response fields:

```
content  
pageNumber  
pageSize  
totalElements  
totalPages  
lastPage
```

19. Swagger Requirement

Swagger UI must be accessible at:

<http://localhost:8080/swagger-ui.html>

OpenAPI JSON must be accessible at:

<http://localhost:8080/v3/api-docs>

Swagger must support Basic Auth authorization.

Swagger must show all Student APIs.

20. Postman Testing Requirement

Base URL:

<http://localhost:8080>

Required APIs to test:

```
POST    /api/students  
GET     /api/students
```

```
GET    /api/students/page?pageNumber=0&pageSize=5
GET    /api/students/{id}
PUT    /api/students/{id}
DELETE /api/students/{id}
GET    /v3/api-docs
GET    /swagger-ui.html
```

Use ADMIN credentials for:

```
POST
PUT
DELETE
```

Use USER or ADMIN credentials for:

```
GET
```

21. Functional Requirements

FR-1: Create Student With Profile

System must allow ADMIN to create a student with exactly one profile.

FR-2: Prevent Duplicate Email

System must not allow two profiles with the same email.

FR-3: Read All Students

System must allow USER and ADMIN to view all students.

FR-4: Read Student By ID

System must allow USER and ADMIN to view one student by ID.

FR-5: Paginated Student List

System must allow USER and ADMIN to fetch students using pagination.

FR-6: Update Student With Profile

System must allow ADMIN to update student and profile details together.

FR-7: Delete Student With Profile

System must allow ADMIN to delete student.

When student is deleted, profile must also be deleted.

FR-8: Validation

System must validate request data before saving or updating.

FR-9: Global Exception Handling

System must return clean and consistent error responses.

FR-10: Role-Based Security

System must restrict APIs based on roles.

FR-11: Swagger Documentation

System must provide browser-based API documentation and testing support using Swagger UI.

22. Non-Functional Requirements

NFR-1: Code Structure

Project must follow layered architecture:

Controller → Service → Repository → Database

NFR-2: DTO Usage

Controller must not directly expose entity classes.

Use DTOs for request and response.

NFR-3: Security

APIs must be secured using Basic Authentication.

JWT must not be used.

NFR-4: Maintainability

Code must be clean, package-wise separated, and easy to extend.

NFR-5: Readability

Use meaningful class names, method names, and field names.

NFR-6: Boilerplate Reduction

Use Lombok to reduce boilerplate code.

NFR-7: Mapping

Use ModelMapper for DTO-to-entity and entity-to-DTO conversion.

23. Development Order

Follow this exact order while building the project:

1. Create Spring Boot project.
 2. Add required Maven dependencies.
 3. Configure application.properties.
 4. Create MySQL database.
 5. Create entity classes.
 6. Create DTO classes.
 7. Create repository interfaces.
 8. Create custom exception classes.
 9. Create global exception handler.
 10. Create AppConfig for ModelMapper.
 11. Create service interface.
 12. Create service implementation.
 13. Create controller.
 14. Add security configuration.
 15. Add Swagger/OpenAPI configuration.
 16. Run application.
 17. Test APIs in Swagger.
 18. Test APIs in Postman.
-

24. Final Expected Output

After completing this project, the application must support:

Student creation with one profile
Student update with profile
Student deletion with profile
Get all students
Get student by id
Pagination
DTO-based API communication
Validation
Global exception handling
Role-based Basic Authentication
Swagger UI documentation
MySQL persistence
One-to-one JPA mapping